

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
AI(Artificial Intelligence)
Practical VII(4,5)

Roll No.:T006	Name:Nandini Chaudhari
Class:TYIT	Batch:01
Date of Assignment:07-08-2026	Date/Time of Submission:07-08-2026

4. Implement k-fold cross-validation on a classification model.

Code(iris Dataset):-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
#Load dataset
iris=load_iris()
x=iris.data
y=iris.target
#Create classification model
model=KNeighborsClassifier()
#Apply 5-fold cross-validation
scores=cross_val_score(model,x,y,cv=5)
print("Accuracy scores for each fold:")
print(scores)
print("Average Accuracy:",round(scores.mean()*100,2,"%"))
print("Nandini Chaudhary T006")
```

Output:-

```
... Accuracy scores for each fold:
[0.96666667 1. 0.93333333 0.96666667 1. ]
Average Accuracy: 97.33 %
Nandini Chaudhary T006
```

Code(California_Housing Dataset):-

```
import pandas as pd
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsRegressor
# Load dataset
data = pd.read_csv("California_Housing.csv")
# Features and Target
X = data.drop("MedHouseVal", axis=1)
y = data["MedHouseVal"]
# Create Regression Model
model = KNeighborsRegressor()
# Apply 5-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=5, scoring="r2")
print("R² Score for each fold:")
print(scores)
print("Average R² Score:", round(scores.mean(), 4))
print("Nandini Chaudhary T006")
```

Output:-

```
*** Accuracy Score for each fold:
[ -5.88495857 -59.91906722 -0.17697979 -1.48992828 -66.10774044]
Average Accuracy Score: -26.7157
Nandini Chaudhary T006
```

Comparison and Justification

Feature	Iris Dataset Code	California Housing Dataset Code
Dataset Type	Classification	Regression
Algorithm Used	KNeighborsClassifier	KNeighborsRegressor
Target Variable	Iris flower species (categorical)	Median House Value (continuous)
Validation Method	5-Fold Cross Validation	5-Fold Cross Validation
Evaluation Metric	Accuracy	R ² Score (default score for regression)
Purpose	Predict flower class	Predict house prices

Conclusion

1) Cross-validation improves the reliability of model evaluation.

2) KNeighborsClassifier is suitable for classification problems like the Iris dataset, while **KNeighborsRegressor** is suitable for regression problems like the California Housing dataset.

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

Code(irisDataset):-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

```
print("Nandini Chaudhary T006")
```

Output:-

```
... Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
Nandini Chaudhary T006
```

(California_Housing)

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
# Load dataset
data = pd.read_csv("California_Housing.csv")
# Features and Target
X = data.drop("MedHouseVal", axis=1)
y = data["MedHouseVal"]
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# Create model
model = DecisionTreeRegressor()
# Train model
model.fit(X_train, y_train)
# Display accuracy (R2 Score)
print("Training Score:", model.score(X_train, y_train) * 100)
print("Testing Score:", model.score(X_test, y_test) * 100)
# Prediction
prediction = model.predict([X_test.iloc[0]])
print("Predicted House Value:", prediction[0])
print("Nandini Chaudhary T006")
```

Output:-

```
... Training Score: 100.0
Testing Score: -21.973423961929804
Predicted House Value: 4.526
Nandini Chaudhary T006
```

Feature	Iris Dataset Code	California Housing Dataset Code
Dataset Type	Classification	Regression
Algorithm Used	DecisionTreeClassifier	DecisionTreeRegressor
Target Variable	Iris flower species (categorical)	Median House Value (continuous)
Data Split	70% Training, 30% Testing	70% Training, 30% Testing
Evaluation Metric	Accuracy	R ² Score
Purpose	Predict flower class	Predict house prices

Conclusion

- 1) A 70:30 train-test split is used to train and evaluate the model on both datasets.
- 2) `DecisionTreeClassifier` is suitable for classification problems like the Iris dataset, while `DecisionTreeRegressor` is suitable for regression problems like the California Housing dataset.